ปลายภาค 1/69 · การเขียนโปรแกรมเชิงวัตถุ (Java)

# สรุปเตรียมสอบ บทที่ 7–12 ฉบับอ่านจบในคืนเดียว

วิเคราะห์จากสไลด์ของอาจารย์ครบทั้ง 6 บท และวิเคราะห์จากแนวข้อสอบที่อาจารย์ประกาศ ทุกหน้าเน้นเฉพาะจุดที่น่าจะออกสอบและจุดที่ถูกหลอกบ่อย

7 Array & ArrayList | 8 Inheritance | 9 Abstraction | 10 Polymorphism | 11 Exception | 12 Java GUI

วิเคราะห์แนวข้อสอบ

# ข้อสอบมี 2 ตอน และแต่ละตอนวัดคนละทักษะ

## ตอนที่ 1 · ปรนัย 4 ตัวเลือก 30 ข้อ

ถามเรื่องหลักการ แนวคิดการออกแบบ และ**ผลการรันโค้ด** ของบท 7–12

- เฉลี่ยประมาณ **5 ข้อต่อบท**
- โจทย์ “ผลการรันโค้ด” = ต้องไล่โค้ดในหัวได้ เช่น index, `super()`, ลำดับ catch/finally, ตัวแปร parent ชี้ object ลูก
- โจทย์แนวคิด = นิยาม + ตารางเปรียบเทียบ (overload/override, abstract/interface, checked/unchecked, throw/throws)
- โจทย์ออกแบบ = เลือกว่าควรใช้ IS-A หรือ HAS-A, abstract class หรือ interface, Layout ตัวไหน

## ตอนที่ 2 · เขียนโค้ด

### 2.1 เขียนคลาสจาก UML Class Diagram

ครอบคลุม Inheritance, Polymorphism, Abstraction ซึ่งตรงกับแบบฝึกหัด Pet/Dog/Cat/Bird, Employee และ Shape ในสไลด์

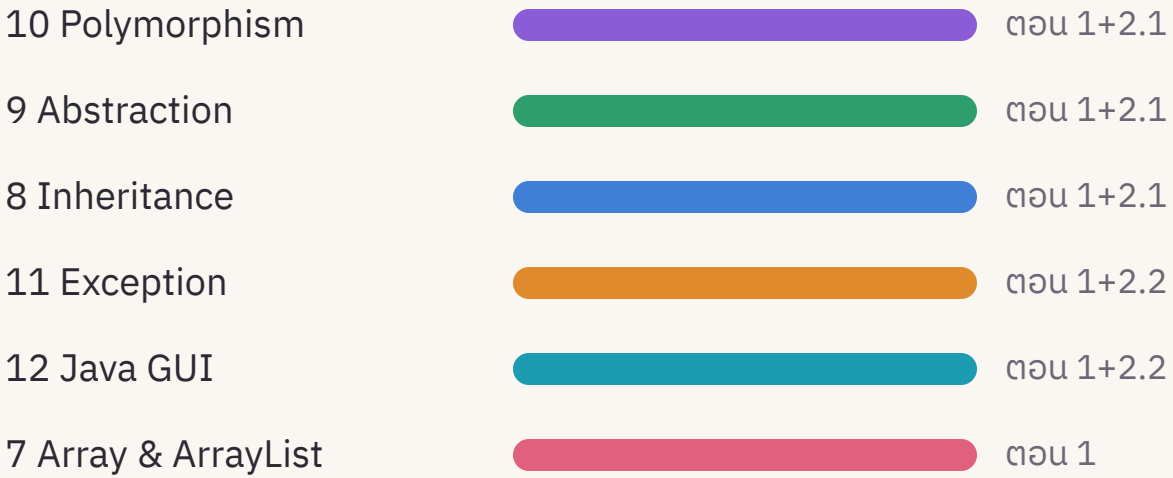
### 2.2 เติมโค้ดส่วนที่หายไป

ครอบคลุม Exception และ GUI ซึ่งตรงกับตัวอย่าง AdditionGUI, GreetingGUI, StudentForm และเครื่องคิดเลข try-catch

**สรุป:** ตอน 2 แทบจะลอกโครงจากตัวอย่างในสไลด์มาทั้งหมด ถ้าเขียนโค้ด 4 ตัวอย่างนั้นได้เองจนคล่อง ก็ได้คะแนนตอนนี้เกือบเต็ม

วิเคราะห์แนวข้อสอบ

# ควรอ่านอะไรก่อน ลำดับความสำคัญ (ประเมินจากรูปแบบข้อสอบ)




บท 7 ไม่ได้อยู่ในตอน 2 โดยตรง แต่ `ArrayList<Pet>` , `Shape[]` และลูป for-each จะโผล่ในโค้ดตอน 2 แน่นอน

### บท 8+9+10 คือเรื่องเดียวกัน

UML หนึ่งรูปทดสอบทั้ง 3 บทพร้อมกัน ได้แก่

- **extends + super(...)** → บท 8
- **abstract method ที่ลูกต้อง override** → บท 9
- **ตัวแปรแม่ชี้ไปที่ object ลูก แล้ววนลูปเรียกเมธอด** → บท 10

บท 11+12 ก็มาด้วยกันเหมือนกัน ตัวอย่างคือปุ่มคำนวณใน GUI ที่ครอบ `parseDouble` ด้วย `try / catch (NumberFormatException)` แล้วเด้ง `JOptionPane`

# 07

# Array & ArrayList

Array 1 มิติ, 2 มิติ, Array of Objects และ ArrayList พร้อม Exception ที่มักเกิดคู่กัน

บทที่ 7

# Array: ขนาดคงที่ และ index เริ่มที่ 0

```java
int[] a = new int[5];          // ค่าเริ่มต้น 0 ทุกช่อง
int[] b = {75, 88, 92, 60, 78};
int[] c = new int[]{75, 88, 92};

for (int i = 0; i < b.length; i++) { ... }
for (int s : b) { ... }   // อ่านอย่างเดียว
```

| ชนิด | ค่าเริ่มต้น |
|---|---|
| `int` / `double` | `0` / `0.0` |
| `boolean` | `false` |
| `String` และ object ทุกชนิด | `null` |

## 5 จุดที่โดนหลอกบ่อย

1. `a.length` **ไม่มีวงเล็บ** (มันเป็น attribute) ส่วน `String.length()` และ `list.size()` มีวงเล็บ
2. index สุดท้ายคือ `length-1` ถ้าเขียน `i <= a.length` จะได้ **ArrayIndexOutOfBoundsException**
3. เทียบ String ต้องใช้ `.equals()` **ไม่ใช่** `==`
4. หาค่าเฉลี่ยต้อง cast ก่อน: `(double) sum / n` เพราะ `7/2 = 3` แต่ `(double)7/2 = 3.5`
5. for-each ใช้เมื่อแค่อ่าน ถ้าต้องใช้ index หรือแก้ค่าในช่อง ให้ใช้ for ปกติ

บทที่ 7

# Array 2 มิติ และ Array of Objects

```
int[][] s = {{75,80,70},
             {88,70,95},
             {92,85,88}};
s.length      // 3 = จำนวนแถว
s[0].length   // 3 = จำนวนคอลัมน์
s[1][2]       // 95  ลำดับคือ [แถว][คอลัมน์]

for (int r = 0; r < s.length; r++)
  for (int c = 0; c < s[r].length; c++)
    ...
```

Jagged array (แต่ละแถวยาวไม่เท่ากัน) ต้องใช้ `s[r].length` ในลูปใน ห้ามใช้ `s.length`

```
Student[] st = new Student[3]; // มีแต่ null 3 ช่อง!
st[0].showInfo();      // × NullPointerException

st[0] = new Student("สมชาย", 3.25); // ต้อง new ทีละตัว
if (st[i] != null) st[i].showInfo();
```

| | Array ของ int | Array ของ object |
|---|---|---|
| เก็บอะไร | ค่าโดยตรง | **reference** (ที่อยู่) |
| ค่าเริ่มต้น | `0` | `null` |
| error ที่เจอบ่อย | IndexOutOfBounds | **NullPointer** |

บทที่ 7

# ArrayList: ขยายขนาดเองได้ (`import java.util.ArrayList;`)

```
ArrayList<String> n = new ArrayList<>();
n.add("A"); n.add("B"); n.add("C");
n.add(1, "X");     // แทรก → [A, X, B, C]
n.get(2);          // "B"
n.size();          // 4
n.remove(0);       // ตัวที่เหลือเลื่อนซ้าย → [X, B, C]
n.set(0, "Z");     // แทนที่ → [Z, B, C]
n.contains("B");   // true
n.clear(); n.isEmpty(); // true
```

| | Array | ArrayList |
|---|---|---|
| ขนาด | คงที่ | ขยายเอง |
| นับจำนวน | `.length` | `.size()` |
| ชนิดข้อมูล | primitive ได้ | ต้องใช้ wrapper |

## จุดที่โดนหลอก

1. `ArrayList<int>` ผิด ต้องเขียนเป็น `ArrayList<Integer>` (Double, Character ก็เช่นกัน) โดย Java จะ autobox ให้อัตโนมัติ
2. `list.get(list.size())` ทำให้ index เกิน ตัวสุดท้ายคือ `size()-1`
3. ลบของออกระหว่างวน for-each จะได้ **ConcurrentModificationException** วิธีแก้คือวนย้อนกลับ `for(i=size()-1; i>=0; i--)`
4. ลืม `import java.util.ArrayList;`
5. `println(list)` พิมพ์ได้เป็น `[A, B, C]`

# 08

# Inheritance (การสืบทอด)

extends, super, access modifier, ลำดับการทำงานของ constructor และการ override

บทที่ 8

# IS-A ใช้ extends · HAS-A ใช้ attribute

**IS-A** → สืบทอดได้: Student *is a* Person, Dog *is a* Animal, Manager *is a* Employee

**HAS-A** → ห้ามสืบทอด ให้ใช้ composition แทน: Car *has an* Engine, CafeOrder *has many* OrderItem

| Modifier | คลาสเดียวกัน | คลาสลูก | คนนอก | UML |
|---|---|---|---|---|
| `public` | ✓ | ✓ | ✓ | + |
| `protected` | ✓ | ✓ | ×* | # |
| `private` | ✓ | × | × | − |

* คลาสใน package เดียวกันเข้าถึง protected ได้ · ถ้าไม่ใส่ modifier เลย (default) จะเข้าถึงได้แค่ใน package เดียวกัน

```
class Student extends Person {
  private String studentId;
  Student(String name, int age, String id) {
    super(name, age);   // ต้องอยู่บรรทัดแรก!
    this.studentId = id;
  }
}
```

## กฎของ super()

- `super(...)` ต้องเป็น **คำสั่งแรก** ใน constructor
- ถ้าไม่เขียน Java จะใส่ `super()` ให้อัตโนมัติ ดังนั้นถ้าคลาสแม่ไม่มี constructor แบบไม่รับพารามิเตอร์ จะ **compile error**
- `new Child()` จะรัน constructor ของ**แม่ก่อน**แล้วค่อยรันของลูก

บทที่ 8

# Override และการเรียก super.method()

## กฎการ Override

- ชื่อเมธอด **เหมือนเดิม** พารามิเตอร์ **เหมือนเดิม** (ทั้งชนิด ลำดับ และจำนวน)
- return type ต้องเหมือนเดิม หรือเป็นชนิดลูก (covariant)
- access modifier **ห้ามแคบลง** (private < default < protected < public)
- **override ไม่ได้:** `static`, `final`, `private` และ constructor (constructor ไม่ถูกสืบทอด)
- static ที่ชื่อซ้ำกันเรียกว่า **method hiding** ไม่ใช่ polymorphism
- ใส่ `@Override` ไว้เสมอ ถ้าพิมพ์ชื่อผิด เช่น `makesound` จะได้ compile error ทันที

```
class Manager extends Employee {
  @Override
  public double calculatePay() {
    double base = super.calculatePay();
    return base + bonus;  // ต่อยอดของแม่
  }
}
```

ถ้าไม่เรียก `super.xxx()` พฤติกรรมของแม่จะ**ถูกแทนที่ทั้งหมด**

Multi-level: `Animal → Dog → Poodle` Poodle ได้ทั้งของ Animal และของ Dog

**ประโยชน์ของ Inheritance:** นำโค้ดกลับมาใช้ซ้ำ (code reuse) ดูแลรักษาง่าย ขยายระบบง่าย และจำลองโลกจริงได้

# 09

# Abstraction

แสดงเฉพาะสิ่งที่จำเป็นและซ่อนรายละเอียด ทำได้ 2 เครื่องมือคือ abstract class และ interface

บทที่ 9

# Abstract class: กฎทองที่ต้องจำ

```
public abstract class Shape {
  private String color;
  public Shape(String color) {   // มี constructor ได้
    this.color = color;
  }
  public abstract double calculateArea(); // ไม่มี body ปิดด้วย ;
  public void displayInfo() {           // เมธอดปกติก็มีได้
    System.out.printf("Area: %.2f%n", calculateArea());
  }
}
Shape s = new Shape("red");       // × new ไม่ได้
Shape s = new Circle("red", 5); // ✓
```

## กฎ

- **new abstract class โดยตรงไม่ได้** แต่ใช้เป็นชนิดของตัวแปรได้
- มีทั้ง attribute, constructor, เมธอดปกติ และ abstract method ได้
- ถ้ามี abstract method **อย่างน้อย 1 ตัว** คลาสนั้นต้องเป็น abstract
- abstract class ที่**ไม่มี** abstract method เลยก็ได้ (ใช้กันไม่ให้ new)
- abstract method จะเป็น `private`, `final` หรือ `static` **ไม่ได้**
- คลาสลูกต้อง override abstract method **ให้ครบทุกตัว** ถ้าไม่ครบ ลูกต้องเป็น abstract ด้วย

UML: ชื่อคลาสตัวเอียงหรือมี «abstract» กำกับ, abstract method มี `{abstract}` กำกับ, ลูกศรสามเหลี่ยมโปร่งเส้นทึบชี้ไปที่คลาสแม่

บทที่ 9

# Interface: สัญญาที่คลาสต้องทำตาม

```
interface PaymentMethod {
  void pay(double amount); // = public abstract อัตโนมัติ
}
class CashPayment implements PaymentMethod {
  @Override
  public void pay(double amount) { ... } // ต้อง public!
}
class Phone implements Camera, MusicPlayer, GPS {}
class FullTime extends Employee implements Taxable {}
interface MFD extends Printable, Scannable {}
```

UML: ใช้ «interface» และ**เส้นประ**หัวสามเหลี่ยมโปร่ง (realization)

## สิ่งที่อยู่ใน interface ได้

- abstract method ซึ่งจะเป็น `public abstract` อัตโนมัติ
- ค่าคงที่ซึ่งจะเป็น `public static final` อัตโนมัติ ดังนั้น `Taxable.VAT_RATE = 0.1;` จะ error
- `default` method (มี body), `static` method (เรียกผ่านชื่อ interface) และ `private` method

## กับดัก

- เขียนเมธอดที่ implement โดยไม่ใส่ `public` จะ error ว่า *weaker access*
- `new PaymentMethod()` × แต่ `PaymentMethod p = new CashPayment()` ✓
- **extends** ได้คลาสเดียว แต่ **implements** ได้หลาย interface และต้องเขียน extends ก่อน implements

บทที่ 9

# Abstract class หรือ Interface?

| ประเด็น | Abstract class | Interface |
|---|---|---|
| คำที่ใช้ | `extends` | `implements` |
| new ได้ไหม | × | × |
| constructor | ✓ | × |
| instance attribute | ✓ | × (มีแต่ค่าคงที่) |
| method ปกติ | ✓ | default / static / private |
| access | อะไรก็ได้ | abstract method เป็น public |
| สืบทอดได้กี่ตัว | 1 คลาส | หลาย interface |
| เหมาะกับ | โครงสร้างร่วม (is-a) | ความสามารถร่วม (-able) |

### เลือก abstract class เมื่อ

คลาสเป็นญาติใกล้ชิดกัน (is-a) มี attribute หรือ constructor ร่วมกัน และมีโค้ดพื้นฐานที่อยากแชร์

### เลือก interface เมื่อ

คลาสที่ไม่เกี่ยวกันต้องมี “ความสามารถ” เดียวกัน หรือต้องการหลายความสามารถ ชื่อมักลงท้ายด้วย -able เช่น Printable, Payable

**Abstraction vs Encapsulation:** abstraction บอกว่า “ทำอะไรได้” (abstract class/interface) ส่วน encapsulation บอกว่า “ปกป้องข้อมูลอย่างไร” ( `private` + getter/setter)

# 10

# Polymorphism (การพ้องรูป)

ตัวแปรชนิดเดียวกันชี้ไปที่ object ต่างชนิดกัน ผลลัพธ์จะขึ้นกับ object จริงที่ถูกชี้

บทที่ 10

# กฎข้อเดียวที่ตอบได้เกือบทุกข้อ

**ชนิดของตัวแปร (reference type)** กำหนดว่า **เรียกเมธอดอะไรได้** ซึ่งตรวจตอน compile

**ชนิดของ object จริง** กำหนดว่า **เมธอดทำงานอย่างไร** ซึ่งตัดสินตอน runtime

```
Animal a = new Dog();
a.makeSound();   // "Dog barks: Woof!"  ← ของ Dog
a.guardHouse();  // × Compile Error (Animal ไม่มี)
a = new Cat();
a.makeSound();   // "Cat meows: Meow!"
```

| | Overloading | Overriding |
|---|---|---|
| ชื่อเมธอด | เหมือน | เหมือน |
| พารามิเตอร์ | **ต่าง** | **เหมือน** |
| ต้องสืบทอด? | ไม่จำเป็น | ต้องมี inheritance หรือ interface |
| return type | ต่างได้ | เหมือนหรือ covariant |
| ตัดสินตอน | **Compile time** | **Runtime** |
| ชื่อเรียก | static / early binding | dynamic method dispatch |

ถ้า**ต่างกันแค่ return type** จะไม่นับเป็น overloading แต่เป็น **compile error**

บทที่ 10

# Upcasting, Downcasting และ instanceof

```
// Upcasting: ลูก → แม่ อัตโนมัติและปลอดภัย
Animal a = new Dog();

// Downcasting: แม่ → ลูก ต้อง cast เอง
Dog d = (Dog) a;      // ✓ เพราะร่างจริงเป็น Dog
d.guardHouse();

Animal x = new Cat();
Dog y = (Dog) x;      // compile ผ่าน แต่รันแล้วได้
                      // ClassCastException

if (a instanceof Dog dog) {  // เช็คชนิดก่อน cast
  dog.guardHouse();
}
```

## สิ่งที่ไม่เกิด polymorphism

```
class Parent { String name="Parent"; ... }
class Child extends Parent { String name="Child"; ... }
Parent o = new Child();
o.name;       // "Parent"  ← field ยึดตามชนิดตัวแปร
o.showName(); // "Child method" ← method ยึดตาม object
```

- **field** และ **static method** ไม่เกิด polymorphism
- **private**, **final** และ **constructor** override ไม่ได้

บทที่ 10

# ใช้ประโยชน์: ลูปเดียวจัดการได้ทุกชนิด

```
Shape[] shapes = {
  new Circle("Red", 5.0),          // 78.54
  new Rectangle("Blue", 4.0, 6.0),  // 24.00
  new Triangle("Green", 8.0, 3.0) // 12.00
};
double total = 0;
for (Shape s : shapes)
  total += s.calculateArea(); // ← polymorphism เกิดที่บรรทัดนี้
System.out.printf("Total area: %.2f%n", total); // 114.54
```

```
void processPayment(PaymentMethod p, double amt) {
  p.pay(amt);    // ไม่ต้องรู้ว่าจ่ายด้วยวิธีไหน
}
```

### Open/Closed Principle

เพิ่มช่องทางจ่ายเงินใหม่ได้ด้วยการสร้างคลาสใหม่ที่ `implements PaymentMethod` โดยไม่ต้องแก้ if-else เดิม (เปิดให้ขยาย แต่ปิดการแก้ไข)

### Program to an interface

ให้เมธอดรับพารามิเตอร์เป็น `checkout(PaymentMethod p)` แทน `checkout(CreditCardPayment p)`

### Code smell

ถ้าเขียน `if (s instanceof Circle) ... else if (s instanceof Rectangle)` เยอะ ๆ แปลว่ายังใช้ polymorphism ไม่คุ้ม ควรย้ายไปไว้ใน `calculateArea()` ของแต่ละคลาส

# 11

# Exception Handling

จัดการข้อผิดพลาดตอนรันเพื่อให้โปรแกรมไม่ตายกลางทาง

บทที่ 11

# ข้อผิดพลาด 3 แบบ และ 5 Exception ยอดฮิต

| แบบ | ลักษณะ |
|---|---|
| **Syntax/Compile** | ผิดหลักภาษา เช่น ลืม `;` ทำให้ compiler หยุดก่อนรัน |
| **Runtime Exception** | โค้ดถูกไวยากรณ์ แต่พังตอนรัน เช่น หาร 0 หรือ index เกิน |
| **Logical** | รันได้ แต่ผลลัพธ์ผิด เช่น ใส่สูตรผิด |

ลำดับชั้นของคลาส: **Throwable** แตกเป็น **Error** และ **Exception** ส่วน Exception มีลูกคือ **RuntimeException** ซึ่งมีลูกเช่น ArithmeticException และ NullPointerException ดังนั้น `catch (Exception e)` จึงรับได้ทุกตัวเพราะ polymorphism

| Exception | เกิดจาก |
|---|---|
| **ArithmeticException** | `10 / 0` (หาร int ด้วยศูนย์) |
| **ArrayIndexOutOfBounds** | `score[5]` ทั้งที่มีแค่ 3 ช่อง |
| **NumberFormatException** | `Integer.parseInt("ABC")` หรือ `Double.parseDouble` กับข้อความที่ไม่ใช่ตัวเลข |
| **NullPointerException** | `name = null; name.length()` |
| **InputMismatchException** | `scanner.nextInt()` แต่ผู้ใช้พิมพ์ตัวอักษร (ต้อง import จาก `java.util`) |
| **ClassCastException** | downcast ผิดร่าง (จากบท 10) |

บทที่ 11

# try / catch / finally: ไล่ผลการรันให้ถูก

```
try {
  int[] x = {10, 20, 30};
  System.out.println(x[5]);  // ← พังที่บรรทัดนี้
  int r = 10 / 0;            // ไม่ถูกรันเลย
} catch (ArithmeticException e) {
  System.out.println("A");
} catch (ArrayIndexOutOfBoundsException e) {
  System.out.println("I");
} finally {
  System.out.println("F");
}
System.out.println("End");
// ผลลัพธ์:  I  F  End
```

## กฎที่ใช้ไล่โค้ด

- เมื่อเกิด exception บรรทัดที่เหลือใน `try` จะ**ถูกข้ามทั้งหมด**
- Java จะเลือก catch **ตัวแรกที่ชนิดตรงกัน** แค่ตัวเดียว
- `finally` **ทำงานเสมอ** ไม่ว่าจะเกิด error หรือไม่ (ใช้ปิดไฟล์หรือคืนทรัพยากร)
- หลังจากจัดการแล้ว โปรแกรมทำงานต่อหลังบล็อก try
- **catch ที่เจาะจงต้องอยู่ก่อน catch ที่กว้างกว่า** ถ้าเอา `catch(Exception e)` ไว้ก่อน catch ลูก จะ **compile error**

## ตัวแปร e

- `e.getMessage()` ได้ `/ by zero`
- `println(e)` ได้ `java.lang.ArithmeticException: / by zero`
- `e.printStackTrace()` ใช้ตอน debug

บทที่ 11

# Checked vs Unchecked · throw vs throws · Custom Exception

| | Unchecked | Checked |
|---|---|---|
| compiler บังคับ? | ไม่บังคับ | **บังคับ** try-catch หรือ throws |
| ตัวอย่าง | Arithmetic, NullPointer, NumberFormat, ArrayIndex, InputMismatch | **FileNotFoundException**, IOException |

| คำสั่ง | หน้าที่ |
|---|---|
| `throw` | **โยน** exception ออกไปทันที (เขียนในเนื้อเมธอด) |
| `throws` | **ประกาศ**ที่หัวเมธอดว่าอาจโยนออกไป ให้คนเรียกจัดการเอง |

```
public Student(String name, int age) {
  if (age < 0)
    throw new IllegalArgumentException(
        "Age cannot be negative");
  ...
}
```

```
public class InsufficientBalanceException
       extends Exception {
  public InsufficientBalanceException(String msg) {
    super(msg);
  }
}
public void withdraw(double amt)
       throws InsufficientBalanceException {
  if (amt > balance)
    throw new InsufficientBalanceException("Insufficient balance");
  balance -= amt;
}
```

# 12

# Java GUI (Swing)

JFrame, Component, Layout Manager, Event และ JOptionPane ซึ่งออกในตอนที่ 2.2 แน่นอน

บทที่ 12

# โครงโปรแกรม GUI ที่ต้องเขียนได้เอง

```java
import javax.swing.*;   // JFrame, JButton, ...
import java.awt.*;      // Layout, Color, Font

public class MyGUI extends JFrame {
  public MyGUI() {
    setTitle("My First GUI");
    setSize(500, 300);              // กว้าง, สูง
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setLocationRelativeTo(null);    // กลางจอ
  }
  public static void main(String[] args) {
    SwingUtilities.invokeLater(() ->
        new MyGUI().setVisible(true)); // ท้ายสุด!
  }
}
```

## จุดที่ถูกถามบ่อย

- `setVisible(true)` ต้องอยู่ **ท้ายสุด** ถ้าลืม หน้าต่างจะไม่แสดง
- `EXIT_ON_CLOSE` ทำให้ปิดหน้าต่างแล้วโปรแกรมหยุดด้วย
- `setLocationRelativeTo(null)` ทำให้หน้าต่างอยู่กลางจอ
- การสร้างและแก้ GUI ต้องทำบน **EDT** ผ่าน `SwingUtilities.invokeLater`
- Swing พัฒนาต่อจาก AWT ส่วน JavaFX เป็นรุ่นใหม่ที่รองรับ CSS
- `pack()` ปรับขนาดหน้าต่างตาม component ซึ่งยืดหยุ่นกว่า `setSize()`
- `dispose()` ทำลายหน้าต่างและคืนหน่วยความจำ ส่วน `setVisible(false)` แค่ซ่อน

บทที่ 12

# Component และเมธอดที่ใช้บ่อย

| Component | ใช้ทำอะไร | โค้ดที่ต้องจำ |
|---|---|---|
| **JLabel** | แสดงข้อความหรือรูป | `new JLabel("ข้อความ", SwingConstants.CENTER)` · `setText(...)` |
| **JTextField** / JPasswordField | รับข้อความบรรทัดเดียว | `new JTextField(20)` โดย 20 คือ**ความกว้าง** ไม่ใช่จำนวนตัวอักษรสูงสุด · `getText().trim()` · `setText("")` |
| **JTextArea** | รับข้อความหลายบรรทัด | `new JTextArea(5, 20)` (แถว, คอลัมน์) · ต้องใส่ใน `new JScrollPane(area)` ถึงจะเลื่อนได้ · `setEditable(false)` |
| **JButton** | ปุ่ม | ปุ่มจะไม่ทำงานจนกว่าจะ `addActionListener` |
| **JCheckBox** | เลือกได้**หลายข้อ** | `isSelected()` · `setSelected(true)` |
| **JRadioButton** | เลือกได้**ข้อเดียว** | **ต้องใส่ใน** `ButtonGroup` แต่ยังต้อง add ลง panel เพื่อจัดวางด้วย |
| **JComboBox<String>** | dropdown | `new JComboBox<>(array)` · `(String) getSelectedItem()` · `getSelectedIndex()` |
| **JTable** | ตาราง | `new JTable(data, columnNames)` ใส่ใน JScrollPane · ถ้าต้องเพิ่มหรือลบแถวใช้ `DefaultTableModel` : `addRow` , `removeRow` , `getSelectedRow()` คืน **-1** ถ้ายังไม่เลือก |
| **JMenuBar** → JMenu → JMenuItem | แถบเมนู | `fileMenu.add(item)` · `addSeparator()` · `menuBar.add(fileMenu)` · `setJMenuBar(menuBar)` |

บทที่ 12

# Layout Manager และการซ้อน JPanel

| Layout | ลักษณะ |
|---|---|
| **FlowLayout** | เรียงจากซ้ายไปขวา ถ้าที่ไม่พอจะขึ้นบรรทัดใหม่ เหมาะกับฟอร์มเล็ก (เป็นค่าเริ่มต้นของ JPanel) |
| **BorderLayout** | แบ่ง 5 ทิศ ได้แก่ NORTH, SOUTH, EAST, WEST, CENTER |
| **GridLayout**(r, c, hgap, vgap) | ตารางที่ทุกช่องขนาดเท่ากัน เช่น `(3, 2, 5, 5)` = 3 แถว 2 คอลัมน์ ระยะห่าง 5 |
| BoxLayout · GridBagLayout | เรียงแนวเดียว · ตารางที่ยืดหยุ่น |

ไม่แนะนำ `setLayout(null)` + `setBounds(...)` เพราะหน้าจอไม่ยืดหยุ่น และพังเมื่อเปลี่ยนเครื่องหรือฟอนต์

```
// ฟอร์มมาตรฐาน: Grid ซ้อนใน Border
JPanel formPanel =
    new JPanel(new GridLayout(2, 2, 5, 5));
formPanel.add(new JLabel("จำนวนที่ 1:"));
formPanel.add(firstField);
formPanel.add(new JLabel("จำนวนที่ 2:"));
formPanel.add(secondField);

JPanel buttonPanel = new JPanel(); // FlowLayout
buttonPanel.add(calculateButton);
buttonPanel.add(clearButton);

setLayout(new BorderLayout(10, 10));
add(formPanel,   BorderLayout.NORTH);
add(resultLabel, BorderLayout.CENTER);
add(buttonPanel, BorderLayout.SOUTH);
```

บทที่ 12

# Event-Driven: Action → Listen → Respond

**Event Source** (JButton) ส่ง **Event Object** (ActionEvent) ไปให้ **Event Listener** (ActionListener)

```
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

// แบบเดิม: anonymous class
button.addActionListener(new ActionListener() {
  @Override
  public void actionPerformed(ActionEvent e) {
    resultLabel.setText("คุณคลิกปุ่มแล้ว");
  }
});

// แบบ Lambda (ใช้ได้เพราะ ActionListener มีเมธอดเดียว)
button.addActionListener(e -> calculate());
```

ItemListener (ComboBox) · MouseListener ใช้คู่กับ `MouseAdapter` ช่วยให้ไม่ต้อง override เมธอดที่ไม่ใช้ · KeyListener ใช้กับ `keyReleased`

| JOptionPane | คืนค่า |
|---|---|
| `showMessageDialog(this, "msg", "title", JOptionPane.ERROR_MESSAGE)` | void |
| `showInputDialog("กรุณากรอกชื่อ")` | **String** หรือ **null** ถ้ากด Cancel |
| `showConfirmDialog(null, "ออก?", "Confirm", JOptionPane.YES_NO_OPTION)` | **int** ให้เทียบกับ `YES_OPTION` |

ชนิดข้อความ: INFORMATION · WARNING · ERROR · QUESTION · PLAIN `_MESSAGE`

## คลินิกแก้บั๊ก

หน้าต่างไม่แสดง → ลืม `setVisible(true)`
กดปุ่มแล้วไม่เกิดอะไร → ลืม `addActionListener`
พิมพ์ตัวอักษรแล้วพัง → ไม่ได้ catch `NumberFormatException`
โปรแกรมค้าง → ทำงานหนักบน EDT ควรใช้ SwingWorker

เตรียมตอนที่ 2.1

# UML → Java: แปลทีละสัญลักษณ์

| ใน UML | เขียนเป็น Java |
|---|---|
| + / # / − | `public` / `protected` / `private` |
| `name : String` | `String name;` (ชนิดอยู่**หน้า**ชื่อ) |
| `makeSound() : void` | `public void makeSound()` |
| ชื่อ/เมธอดตัวเอียง, «abstract», {abstract} | `abstract` |
| «interface» | `interface` |
| เส้นทึบ + สามเหลี่ยมโปร่ง | `extends` |
| เส้นประ + สามเหลี่ยมโปร่ง | `implements` |
| constructor ของลูก | บรรทัดแรกต้องเป็น `super(...)` |

ขั้นตอน: เขียนคลาสแม่ก่อน → ลูกทีละตัว → main ที่ใช้ `ArrayList<แม่>` + for-each

```java
public abstract class Pet {
  protected String name;              // # name : String
  public Pet(String name) {
    this.name = name;
  }
  public abstract void makeSound();
  public void showPetInfo() {
    System.out.print(name + " Making Sound : ");
    makeSound();                          // ← polymorphism
  }
}
class Dog extends Pet {
  public Dog(String name) { super(name); }
  @Override
  public void makeSound() {
    System.out.println("Bok Bok...");
  }
}
// Cat, Bird เขียนแบบเดียวกัน
ArrayList<Pet> pets = new ArrayList<>();
pets.add(new Dog("Buddy"));
pets.add(new Cat("Mimi"));
for (Pet p : pets) p.showPetInfo();
// Buddy Making Sound : Bok Bok...
```

เตรียมตอนที่ 2.2

# เติมโค้ดที่หายไป: ช่องที่มักถูกเว้นให้เติม

```
import javax.swing.*;
import java.awt.*;
public class AdditionGUI extends JFrame {
  ...
  calculateButton.addActionListener(e -> calculate());
  setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
  ...
  private void calculate() {
    try {
      double a = Double.parseDouble(
          firstField.getText().trim());
      double b = Double.parseDouble(
          secondField.getText().trim());
      resultLabel.setText("ผลลัพธ์ = " + (a + b));
    } catch (NumberFormatException e) {
      JOptionPane.showMessageDialog(this,
          "กรุณากรอกข้อมูลเป็นตัวเลข", "ข้อมูลไม่ถูกต้อง",
          JOptionPane.ERROR_MESSAGE);
    }
  }
  public static void main(String[] args) {
    SwingUtilities.invokeLater(() ->
        new AdditionGUI().setVisible(true));
  }
}
```

## คำที่ควรเขียนได้โดยไม่ต้องคิด

`extends JFrame` · `addActionListener` · `getText` · `setText` · `parseInt` / `parseDouble` · `try` / `catch` / `finally` · `NumberFormatException` · `InputMismatchException` · `ArithmeticException` · `throw new IllegalArgumentException(...)` · `throws` · `JOptionPane.showMessageDialog` · `BorderLayout.NORTH` · `new GridLayout(r,c)` · `ButtonGroup` · `isSelected()` · `getSelectedItem()` · `new JScrollPane(...)` · `setVisible(true)`

## Validation ในฟอร์ม (StudentForm)

`if (id.isEmpty() || name.isEmpty()) { showMessageDialog(...WARNING_MESSAGE); return; }`

`return;` ใช้หยุดเมธอดทันทีเมื่อข้อมูลไม่ครบ

ทวนก่อนเข้าห้องสอบ

# ทายผลการรันแบบเร็ว (คำตอบทุกข้อทดสอบรันจริงแล้ว)

| โค้ด | ผลลัพธ์ |
|---|---|
| `new int[3]` แล้ว `a[1]` | `0` |
| `new String[2]` แล้ว `s[0]` | `null` |
| `7/2` กับ `(double)7/2` | `3` กับ `3.5` |
| `[A,B,C]` → `add(1,"X")` → `get(2)` | `B` (size = 4) |
| `{{75,80,70},{88,70,95},…}` → `s[1][2]` | `95` |
| `ArrayList<Integer> [10,20,30]` → `remove(1)` | `[10, 30]` (ลบที่ index) |

| โค้ด | ผลลัพธ์ |
|---|---|
| `new Child()` (แม่กับลูกต่างพิมพ์ใน constructor) | ของ**แม่**ก่อน แล้วค่อยของลูก |
| `Parent o = new Child(); o.name` | ค่าของ **Parent** |
| `o.showName()` (ลูก override) | เมธอดของ **Child** |
| `Animal a = new Cat(); (Dog) a` | ClassCastException |
| `try{10/0}catch(Arith..){"Error"}finally{"Finally"}` | Error → Finally |
| `catch(Exception)` ไว้ก่อน `catch(Arithmetic…)` | **compile error** |

แผนอ่าน

# แผน 2 วันก่อนสอบวันพุธ

## วันจันทร์ · บท 8 9 10

- อ่านสไลด์หน้า 8–18 ของชุดนี้
- **เขียนเองโดยไม่ดูเฉลย:** Pet/Dog/Cat/Bird + เมนู, Employee (FullTime/PartTime), Shape (Circle/Rectangle/Triangle)
- ฝึกวาดตาราง overload/override และ abstract/interface จากความจำ

## วันอังคาร · บท 11 12 7

- เขียน AdditionGUI กับ GreetingGUI เองให้รันได้
- เขียนเครื่องคิดเลข try-catch (Arithmetic + InputMismatch) แล้วลองทั้ง 3 test case
- ทวนเมธอด ArrayList และจุดที่โดนหลอกของ array
- ปิดท้ายด้วยหน้า “ทายผลการรันแบบเร็ว”

## เทคนิคในห้องสอบ

- ข้อทายผล: ไล่ทีละบรรทัดแล้ว**จดค่าตัวแปรลงกระดาษ**
- เจอ `new` abstract/interface, ลืม `super` หรือ catch ผิดลำดับ ให้นึกถึง compile error ก่อน
- ตอน 2: เขียนโครงคลาสให้ครบก่อน แล้วค่อยเติมเนื้อเมธอด เพราะได้คะแนนตามส่วน
- อย่าลืม `@Override`, `import` และ `;`